

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 11
Error Handling



Universitas
Pertamina

Disusun Oleh:
Razqa Azaki 105224046

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Error handling adalah cara untuk menangani kesalahan yang terjadi saat program dijalankan. Tujuannya agar program tidak langsung berhenti ketika muncul error. Kesalahan bisa terjadi karena input yang salah, pembagian dengan nol, atau data yang tidak ditemukan.

Dalam Java, error handling dilakukan dengan beberapa struktur seperti try, catch, finally, throw, dan throws. Dengan menggunakan struktur ini, program dapat memberikan pesan kesalahan yang lebih jelas dan tetap berjalan dengan baik. Karena itu, error handling sangat penting agar program menjadi lebih aman, stabil, dan mudah digunakan. Jenis-jenis error terbagi menjadi dua kesalahan, yaitu Checked Exception adalah kesalahan yang sudah bisa diprediksi oleh kompiler saat program dikompilasi, sedangkan Unchecked Exception adalah kesalahan yang baru terdeteksi saat program dijalankan.

II. Variabel

No.	Nama Variabel	Tipe Data	Fungsi
1.	kodeKereta	String	Menyimpan kode unik untuk setiap kereta
2.	namaKereta	String	Menyimpan nama kereta
3.	rute	String	Menyimpan informasi rute perjalanan kereta
4.	sisakursi	int	Menyimpan jumlah kursi yang tersedia

III. Constructor dan Method

No.	Nama Metode	Jenis Metode	Fungsi
1.	KeretaApi()	Constructor	Menginisialisasi objek kereta dengan kode, nama, rute, dan kapasitas
2.	getKodeKereta()	Method	Mengambil nilai kodeKereta
3.	getNamaKereta()	Method	Mengambil nilai namaKereta
4.	getRute()	Method	Mengambil nilai rute
5.	getSisaKursi	Method	Mengambil nilai sisakursi
6.	kurangKursi(int jumlah)	Method	Mengurangi sisa kursi sejumlah tiket yang dipesan
7.	SistemReservasi()	Constructor	Menginisialisasi sistem dan mengisi daftar kereta

BAB II ISI & PEMBAHASAN CODE

```
1 //Unchecked Exception untuk mengecek NIK penumpang
2 public class DataPenumpangTidakValidException extends RuntimeException {
3     public DataPenumpangTidakValidException(String message) {
4         super(message);
5     }
6 }
```

Kelas ini digunakan untuk menangani masalah ketika data penumpang yang dimasukkan tidak valid, yaitu ketika penumpang menginput NIK yang tidak sesuai dengan 16 digit. Karena kelas ini bersifat unchecked, exception ini tidak wajib dideklarasikan dengan throw, namun tetap bisa ditangkap menggunakan catch.

```

1 import java.util.InputMismatchException;
2 import java.util.Scanner;
3
4 public class JavaExpress {
5     public static void main(String[] args) {
6         SistemReservasi reservasi = new SistemReservasi();
7         Scanner input = new Scanner(System.in);
8         boolean berjalan = true;
9
10        try {
11            //Perulangan menu utama selama program masih berjalan
12            while (berjalan) {
13                System.out.println("\n=== SISTEM RESERVASI JAVA EXPRESS ===");
14                System.out.println("1. Lihat Jadwal Kereta");
15                System.out.println("2. Pesan Tiket");
16                System.out.println("3. Keluar");
17                System.out.print("Pilih menu (1-3): ");
18
19                int menu = 0;
20                try {
21                    //Membaca input dari pengguna
22                    menu = input.nextInt();
23                    input.nextLine(); // Membersihkan sisa buffer dari memori
24                } catch (InputMismatchException e) {
25                    //Menangani error jika pengguna salah memasukkan huruf dan lain sebagainya
26                    System.out.println("Error: Input yang anda masukkan salah. Harap masukkan angka.");
27                    input.nextLine(); // Membersihkan buffer error agar tidak infinite loop
28                    continue;
29                }
30
31                switch (menu) {
32                    case 1:
33                        //Menampilkan daftar jadwal kereta
34                        reservasi.tampilkanJadwal();
35                        break;
36                    case 2:
37                        //Memanggil method untuk proses pemesanan tiket kereta
38                        prosesMenuPesan(reservasi, input);
39                        break;
40                    case 3:
41                        //Menghentikan program
42                        berjalan = false;
43                        break;
44                    default:
45                        System.out.println("Pilihan tidak tersedia. Silakan coba lagi.");
46                }
47            }
48        } finally {
49            //Ketika program sudah selesai akan menginput ini diakhir
50            System.out.println("Terima kasih telah menggunakan JAVA EXPRESS. Sampai jumpa!");
51            input.close(); //Menutup scanner
52        }
53    }
54
55    private static void prosesMenuPesan(SistemReservasi reservasi, Scanner input) {
56        System.out.println("\nPESANAN TIKET");
57        try {
58            System.out.print("Masukkan Kode Kereta: ");
59            String kode = input.nextLine();
60
61            System.out.print("Masukkan NIK: ");
62            String nik = input.nextLine();
63
64            System.out.print("Masukkan Nama: ");
65            String nama = input.nextLine();
66
67            System.out.print("Jumlah Tiket: ");
68            int jumlah = input.nextInt();
69            input.nextLine();
70
71            System.out.print("Konfirmasi: Pesan " + jumlah + " tiket untuk " + nama + " (Y/N)? ");
72            String konfirmasi = input.nextLine();
73            //Kondisi jika pengguna memilih "Y" maka pemesanan diproses, jika pengguna memilih "N" maka pemesanan dibatalkan secara otomatis
74            if (konfirmasi.equalsIgnoreCase("Y")) {
75                reservasi.pesanTiket(kode, nik, nama, jumlah);
76                System.out.println("SUKSES! Tiket atas nama " + nama + " berhasil dipesan.");
77            } else {
78                System.out.println("Pemesanan dibatalkan oleh pengguna.");
79            }
80
81        } catch (InputMismatchException e) {
82            System.out.println("Error: Jumlah tiket harus berupa angka.");
83            input.nextLine();
84        } catch (DataPenumpangTidakValidException e) {
85            System.out.println("Error Validasi Data: " + e.getMessage()); //Menangani error jika data penumpang tidak valid
86        } catch (RuteTidakDitemukanException | TiketHabisException e) {
87            System.out.println("Error Reservasi: " + e.getMessage()); //Menangani error jika kode kereta tidak ditemukan atau tiket sudah habis
88        } catch (Exception e) {
89            System.out.println("Terjadi kesalahan sistem yang tidak diketahui: " + e.getMessage()); //Menangani error lain yang tidak terduga
90        }
91    }
92 }

```

Dalam kelas java express adalah kelas utama yang menjalankan seluruh program. Di sini terdapat method main() yang menampilkan menu dan method prosesMenuPesan() yang menangani alur pemesanan tiket. Semua exception yang dilempar oleh SistemReservasi ditangkap di sini menggunakan blok try-catch bertingkat. Blok finally digunakan untuk memastikan Scanner selalu ditutup dan pesan penutup selalu ditampilkan meskipun terjadi error.

```
1 //Checked Exception untuk pencarian kode kereta yang mengalami kesalahan
2 public class RuteTidakDitemukanException extends Exception {
3     public RuteTidakDitemukanException(String message) {
4         super(message);
5     }
6 }
```

Kelas ini digunakan ketika pengguna memasukkan kode kereta yang tidak terdaftar dalam sistem. Karena bersifat checked, method yang melemparnya wajib mendeklarasikan throws RuteTidakDitemukanException.

```
1 public class KeretaApi {
2     private String kodeKereta;
3     private String namaKereta;
4     private String rute;
5     private int sisaKursi;
6
7     public KeretaApi(String kodeKereta, String namaKereta, String rute, int kapasitas) {
8         this.kodeKereta = kodeKereta;
9         this.namaKereta = namaKereta;
10        this.rute = rute;
11        this.sisaKursi = kapasitas;
12    }
13
14    public String getKodeKereta() { return kodeKereta;}
15    public String getNamaKereta() { return namaKereta;}
16    public String getRute() { return rute; }
17    public int getSisaKursi() { return sisaKursi;}
18    public void kurangiKursi(int jumlah) { this.sisaKursi -= jumlah;}
19 }
```

Kelas ini memiliki empat atribut private yaitu kodeKereta, namaKereta, rute, dan sisakursi. Semua atribut bersifat private sesuai prinsip enkapsulasi, sehingga hanya bisa diakses melalui method getter.

Constructor kelas ini menerima empat parameter untuk menginisialisasi semua atribut. Terdapat juga method kurangiKursi() yang digunakan untuk mengurangi sisa kursi ketika tiket berhasil dipesan.

```
1 import java.util.HashMap;
2 import java.util.Map;
3
4 public class SistemReservasi {
5     private Map<String, KeretaApi> daftarKereta;
6
7     public SistemReservasi() {
8         daftarKereta = new HashMap<String, KeretaApi>();
9         //daftar kereta menggunakan kunci
10        daftarKereta.put("K01", new KeretaApi("K01", "Argo Bromo", "3KT - SBV", 50));
11        daftarKereta.put("K02", new KeretaApi("K02", "Parahyangan", "3KT - BDC", 15));
12    }
13
14    public void tampilkanJadwal() {
15        System.out.println("=== Jadwal Kereta ===");
16        for(KeretaApi kereta: daftarKereta.values()){
17            System.out.println("Kode Kereta: " + kereta.getKodeKereta());
18            System.out.println("Kode: " + kereta.getKodeKereta() + "|" + "Nama Kereta: " + kereta.getNamaKereta() + "|" + "Rute: " + kereta.getRute() + "|" + "Kapasitas: " + kereta.getSisaKursi());
19        }
20    }
21
22    //Metode transaksi reservasi dengan validasi ketat
23    public void pesanTiket(String kode, String nik, String nama, int jumlah)
24        throws RuteTidakDitemukanException, TiketHabisException {
25
26        //Validasi NIK
27        if (nik.length() != 16) {
28            throw new DataPenumpangTidakValidException("NIK tidak valid! Harus tepat 16 digit angka.");
29        }
30
31        //Pencarian Kereta menggunakan HashMap
32        KeretaApi keretaDipesan = daftarKereta.get(kode);
33
34        //Jika kereta tidak ditemukan di HashMap, maka gagal akan dilempar ke Exception
35        if (keretaDipesan == null) {
36            throw new RuteTidakDitemukanException("Gagal: Kereta dengan kode " + kode + " tidak ditemukan.");
37        }
38
39        //Mengecek sisa kursi
40        if (jumlah > keretaDipesan.getSisaKursi()) {
41            throw new TiketHabisException(keretaDipesan.getNamaKereta(), keretaDipesan.getSisaKursi());
42        }
43
44        // Jika semua mengecek semua berhasil, maka jumlah kursi dikurangi
45        keretaDipesan.kurangiKursi(jumlah);
46        //Konfirmasi Pemesanan
47        System.out.println("=== Konfirmasi Pemesanan ===");
48        System.out.println("Nama Penumpang: " + nama);
49        System.out.println("NIK: " + nik);
50        System.out.println("Kereta: " + keretaDipesan.getNamaKereta() + " (" + keretaDipesan.getKodeKereta() + ")");
51        System.out.println("Rute: " + keretaDipesan.getRute());
52        System.out.println("Jumlah Tiket: " + jumlah);
53        System.out.println("Sisa Kursi: " + keretaDipesan.getSisaKursi());
54    }
55 }
```

Di sini semua data kereta disimpan menggunakan HashMap dengan kode kereta sebagai kunci (key). Kelas ini memiliki dua method utama yaitu tampilkanJadwal() untuk menampilkan semua kereta, dan pesanTiket() untuk memproses pemesanan. Method pesanTiket() menerapkan validasi berlapis. Pertama, NIK dicek apakah panjangnya tepat 16 karakter jika tidak, DataPenumpangTidakValidException dilempar. Kedua, kode kereta dicari di HashMap jika tidak ditemukan, RuteTidakDitemukanException dilempar. Ketiga, sisa kursi dicek jika tidak mencukupi, TiketHabisException dilempar. Jika semua validasi lolos, pemesanan berhasil diproses. Kemudian, untuk meningkatkan program saya menambahkan konfirmasi pesanan untuk mencegah kesalahan pemesanan tiket kereta.

```

1 //Checked Exception jika kapasitas kursi tidak cukup
2 public class TiketHabisException extends Exception {
3     private String namaKereta;
4     private int sisaKursi;
5
6     public TiketHabisException(String namaKereta, int sisaKursi) {
7         super("Tiket habis untuk kereta " + namaKereta + ". Saat ini kursi hanya tersisa: " + sisaKursi + ".");
8         this.namaKereta = namaKereta;
9         this.sisaKursi = sisaKursi;
10    }
11
12    public String getNamaKereta() { return namaKereta; }
13    public int getSisaKursi() { return sisaKursi; }
14 }

```

Kelas ini menyimpan dua atribut tambahan yaitu namaKereta dan sisakursi. Hal ini berguna agar ketika exception ini ditangkap, informasi tentang kereta mana yang kehabisan tiket dan berapa sisa kursinya bisa langsung diakses. Pesan error dibuat langsung di constructor menggunakan super() yang menggabungkan nama kereta dan sisa kursi menjadi satu kalimat yang informatif.

Output pada program

```
=== SISTEM RESERVASI JAVA EXPRESS ===
1. Lihat Jadwal Kereta
2. Pesan Tiket
3. Keluar
Pilih menu (1-3): 1
=== Jadwal Kereta ===
Kode Kereta: K02
Kode: K02|Nama Kereta: Parahyangan|Rute: JKT - BDG|Kapasitas: 15
Kode Kereta: K01
Kode: K01|Nama Kereta: Argo Bromo|Rute: JKT - SBY|Kapasitas: 50

=== SISTEM RESERVASI JAVA EXPRESS ===
1. Lihat Jadwal Kereta
2. Pesan Tiket
3. Keluar
Pilih menu (1-3): 2

PEMESANAN TIKET
Masukkan Kode Kereta: K01
Masukkan NIK: 1122334455667788
Masukkan Nama: Razqa
Jumlah Tiket: 2
Konfirmasi: Pesan 2 tiket untuk Razqa (Y/N)? Y
=== Konfirmasi Pemesanan ===
Nama Penumpang: Razqa
NIK: 1122334455667788
Kereta: Argo Bromo (K01)
Rute: JKT - SBY
Jumlah Tiket: 2
Sisa Kursi: 48
SUKSES! Tiket atas nama Razqa berhasil dipesan.

=== SISTEM RESERVASI JAVA EXPRESS ===
1. Lihat Jadwal Kereta
2. Pesan Tiket
3. Keluar
Pilih menu (1-3): 3
Terima kasih telah menggunakan JAVA EXPRESS. Sampai jumpa!
```

BAB III KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa error handling merupakan teknik penting dalam pemrograman untuk menangani kesalahan yang muncul saat program dieksekusi. Melalui penerapan error handling, program dapat tetap berjalan dengan baik meskipun terjadi exception, sehingga sistem menjadi lebih aman dan tidak mudah berhenti secara mendadak.

Selain itu, penggunaan struktur try, catch, finally, throw, dan throws memudahkan programmer dalam mengontrol alur program ketika terjadi kesalahan. Dengan demikian, error handling tidak hanya berfungsi untuk memperbaiki kesalahan, tetapi juga untuk meningkatkan kualitas, keamanan, dan kenyamanan penggunaan program.

BAB IV DAFTAR PUSTAKA

[1] Polsri. (n.d.). BAB 12 Dasar Exception Handling
<https://miftakulamin.polsri.ac.id/assets/jeni/JENI-Intro1-Bab12.pdf>